

Prompts

Primer prompt:

Actúa como asistente de programación para una práctica de Estructura de Datos en Python.

Contexto:

Ya tengo un proyecto desarrollado en clase con Python, programación orientada a objetos y una

interfaz gráfica creada con Qt Designer. En el proyecto ya existe una clase Stack con los métodos

básicos de una pila. También existe una ventana o formulario donde se capturan datos y se muestran

resultados, así como la clase que realiza la carga y la detección del evento para convertir la

expression.

Objetivo:

Necesito agregar al proyecto existente la funcionalidad para convertir una expresión infija a

expresión posfija utilizando la estructura de pila ya creada en clase. Necesito se genere una clase

que contenga una pila y un método para llevar acabo la conversion.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces la clase Stack existente. Debes utilizarla e integrarte a su forma actual de trabajo.
3. Respeta la organización actual de archivos, clases, nombres de métodos y estilo de programación del proyecto.

4. Si necesitas proponer cambios, indícalos como cambios mínimos y explica exactamente en qué archivo y en qué parte deben colocarse.
5. La solución debe estar hecha con programación orientada a objetos.
6. La interfaz gráfica debe seguir usando el diseño generado con Qt Designer; no cambies el archivo .ui salvo que sea estrictamente necesario.
7. La aplicación debe permitir capturar una expresión infija, convertirla a posfija y mostrar el resultado en la interfaz.
8. Se asume que la expression infija capturada es válida ya que de otra forma la conversion puede fallar.
9. Usa operadores básicos: +, -, *, /, \$(Potencia) y paréntesis.
10. Explica brevemente cómo funciona el algoritmo y cómo se utiliza la pila en la conversion, para esto genera un archivo .md que contenga esta explicación.

Código existente:

```
[from estructuras.lineales.nodo import Node
```

```
class Stack(object):
```

```
    def __init__(self):
```

```
        self.head = None # Representa el 'Top' de la pila
```

```
    def is_empty(self):
```

```
        return self.head is None
```

```
    def push(self, data):
```

```
        """Inserta un elemento en la parte superior (Top)"""
```

```
        nuevo_nodo = Node(data)
```

```
nuevo_nodo.next = self.head
```

```
self.head = nuevo_nodo
```

```
def pop(self):
```

```
    """Elimina y retorna el elemento en la parte superior (Top)"""
```

```
    if not self.head:
```

```
        return None
```

```
    data = self.head.data
```

```
    self.head = self.head.next
```

```
    return data
```

```
def top(self):
```

```
    return self.head.data if self.head else None
```

```
def print_stack(self):
```

```
    if not self.head:
```

```
        print("Stack: Empty")
```

```
    return
```

```
nodos = []
```

```
actual = self.head
```

```
while actual:
```

```
    nodos.append(str(actual.data))
```

```
    actual = actual.next
```

```
print("Top -> " + " -> ".join(nodos) + " -> Base")
```

```

]
[import sys
from PyQt5.QtWidgets import QApplication
# Importamos tu clase LoadMenu desde su archivo correspondiente
from load.load_menu import LoadMenu

def main():
    # 1. Inicializamos la aplicación de PyQt5
    app = QApplication(sys.argv)

    # 2. Instanciamos el menú principal (tu clase con super().__init__())
    menu = LoadMenu()

    # 3. Mostramos el menú en pantalla
    menu.show()

    # 4. Arrancamos el bucle de eventos de la aplicación
    sys.exit(app.exec_())

if __name__ == "__main__":
    main()
]
[ uic.loadUi("ui/posfija.ui", self)
]

```

Resultado esperado de tu respuesta: - Primero analiza la estructura del código existente.

- Después propón la clase con el método para convertir de infija a posfija.**
- Luego indica cómo conectar ese método con el botón o evento de la interfaz gráfica.**
- Finalmente proporciona pruebas para comprobar que funciona.**

- No omitas explicación, porque debo entender el código para defenderlo en clase ante el profesor y mis compañeros.

-----RESULTADO-----

```
# converter.py
```

```
from estructuras.lineales.stack import Stack # Cambia 'tu_archivo_de_pila' por el nombre real de tu archivo (.py)
```

```
class ExpressionConverter:
```

```
    def __init__(self):
```

```
        # Definimos la precedencia matemática de los operadores
```

```
        self.precedence = {
```

```
            '+': 1,
```

```
            '-': 1,
```

```
            '*': 2,
```

```
            '/': 2,
```

```
            '$': 3 # Potencia
```

```
        }
```

```
    def _is_operator(self, char: str) -> bool:
```

```
        """Método auxiliar interno para identificar operadores."""
```

```
        return char in self.precedence
```

```

def infix_to_postfix(self, infix_expr: str) -> str:
    """Convierte una expresión infija válida a posfija usando la clase Stack de
    clase."""

    operator_stack = Stack()
    postfix_result = []

    # Quitamos espacios en blanco para procesar limpiamente caracter por
    caracter
    infix_expr = infix_expr.replace(" ", "")

    for char in infix_expr:
        # Caso 1: Es un operando (Letra o Número) -> Va directo al resultado
        if char.isalnum():
            postfix_result.append(char)

        # Caso 2: Paréntesis de apertura -> Se guarda en la pila como frontera
        elif char == '(':
            operator_stack.push(char)

        # Caso 3: Paréntesis de cierre -> Desapilar hasta encontrar el '('
        elif char == ')':
            while not operator_stack.is_empty() and operator_stack.top() != '(':
                postfix_result.append(operator_stack.pop())
            operator_stack.pop() # Sacamos el '(' sobrante de la pila y lo
            descartamos

```

```

# Caso 4: Es un operador (+, -, *, /, $)
elif self._is_operator(char):
    # Mientras la pila no esté vacía, no sea un '(' y el operador en el 'top'
    # tenga mayor o igual prioridad que el operador actual, se desapila.
    while (not operator_stack.is_empty() and
           operator_stack.top() != '(' and
           self.precedence.get(operator_stack.top(), 0) >=
self.precedence[char]):
        postfix_result.append(operator_stack.pop())

    # Al terminar la condición, el operador actual se guarda en la pila
    operator_stack.push(char)

# Caso final: Vaciar todo lo que haya quedado rezagado en la pila al
terminar la expresión
while not operator_stack.is_empty():
    postfix_result.append(operator_stack.pop())

return "".join(postfix_result)

```

load_conversion.py

import sys

```

from PyQt5 import QtWidgets, uic

# Importamos la lógica que creamos previamente
from converter import ExpressionConverter

class VentanaConversion(QtWidgets.QDialog): # <-- Cambiado a QDialog para coincidir
con tu .ui

    def __init__(self):

        # 1. Inicializa el constructor padre de QDialog
        super().__init__()

        # 2. Carga los componentes del archivo .ui directamente
        # Cambia 'tu_diseño.ui' por el nombre real de tu archivo de Qt Designer
        uic.loadUi('tu_diseño.ui', self)

        # 3. Instancia de la lógica orientada a objetos (La Pila)
        self.converter = ExpressionConverter()

        # 4. Conectamos el botón usando su nombre exacto del XML: 'pushButton'
        self.pushButton.clicked.connect(self.procesar_conversion)

    def procesar_conversion(self):

        """Lee el input del 'lineEdit', invoca la pila y muestra el resultado en 'label_2'."""

        # Recuperamos la expresión infija usando el name real
        infija = self.lineEdit.text()

        # Validación básica por si el campo está vacío
        if not infija.strip():

            self.label_2.setText("Ingresa una expresión")

            return

```



```
# Ejecutamos la conversión que utiliza la estructura Stack
```

```
posfija = self.converter.infix_to_postfix(infija)
```

```
# Mostramos la conversión en la etiqueta real asignada por Qt: 'label_2'
```

```
self.label_2.setText(posfija)
```

Segundo prompt:

Hazme la funcion para mi archivo de menuLoad para agregar el apartado de la postfija gráficamente

```
def abrir_posfija(self):
```

```
    """Instancia autónoma con super().__init__() para el convertidor."""
```

```
    dialogo = VentanaConversion()
```

```
    dialogo.exec_()
```